

DWS Persistence Unified Guide

Date: 2026-05-29

Status: Verified from existing persistence docs

Table of Contents

- 1. [Overall Cleanup Strategy](#)
- 2. [Known Issues](#)
- 3. [Storage Paths](#)
- 4. [Session Data Cleanup \(Time-Based\)](#)
- 5. [Transaction Persistence \(Event-Driven\)](#)
- 6. [Lexmark Integration](#)

1) Overall Cleanup Strategy

- Session data: time-based cleanup every 600s, retention 3 days.
- Transactions: event-driven remove on job completion.
- Scan files: explicit cleanup when transfer/workflow ends.
- Lexmark does not override cleanup policy; it uses DWS core behavior.

Data type	Cleanup mechanism	Trigger	Result
Session Data	Time-based	Every 600s	Delete folders older than 3 days
Transactions	Event-driven	<code>RemovePersistenceRequest</code> after successful post	Delete <code>{jobUuid}.transaction</code> file
Scan Files	Explicit	<code>PushScanSuccess</code> / <code>PushScanCancel</code> / framework stop	Delete cache and temp scan files

Persistence Chain Index

Chain	Name	Entry Event
P1	Session persist	<code>ISessionEvents.OnLoginEvent</code>
P2	OnLogoutEvent → clear in-memory state (disk files NOT deleted)	<code>ISessionEvents.OnLogoutEvent</code>
P3	Session scheduled cleanup (disk)	<code>CleanUpTransactionBaseTask.run()</code> every 600s
P4	Scan transaction persistence	<code>IDeviceJobEvents.Scan.Begin</code> + <code>IScanTransferManager.PushScanRequest</code>
P5	Copy transaction persistence	<code>IDeviceJobEvents.Copy.EscrowRequest</code>
P6	Print transaction persistence	<code>ITransactionJobPostingProvider.Print.SerializeRequest</code>

2) Known Issues

Issue #1: LostTransaction files are never deleted

Affected path: `LostTransaction/{deviceId}/COPY|SCAN|PRINT-{jobUuid}`

Root cause:

When a transaction posting fails after `MAX_RETRYTIMES` (3 retries),

`LostTransactionsController` calls `persistLostJob()`, which triggers `PersistLostTransactionRequest`.

`TransactionPersistenceHandler.onPersistLostTransactionRequestEvent()` then **renames** (not deletes) the file from `Transaction/{TYPE}/{jobUuid}.transaction` → `LostTransaction/{deviceId}/{TYPE}-{jobUuid}`.

There is **no cleanup mechanism** for the `LostTransaction/` directory:

- No scheduled cleanup task (unlike Session Data which runs every 600s).
- No event-driven deletion (unlike Transaction files which are deleted on `PostResponseSuccess`).
- No TTL or size limit policy.

Impact:

Over time, every failed transaction permanently accumulates a file under `LostTransaction/` . On devices with high transaction failure rates (e.g., prolonged server offline), this can exhaust disk space and potentially impact the entire DWS application.

Affected flow: P7.5 — `retryTimes >= 3` → rename transaction to `LostTransaction` file

Code:

```
PostResponseFailure (retry 1) → PostResponseFailure (retry 2) → PostResponseFailure
→ LostTransactionsController.persistLostJob()
→ TransactionPersistenceHandler.writeLostJob()
    → rename Transaction/COPY|SCAN|PRINT/{jobUuid}.transaction
        → LostTransaction/{deviceId}/COPY|SCAN|PRINT-{jobUuid}
[file stays on disk indefinitely – no deletion code path exists]
```

3) Storage Paths

- Session data: `HistoricalSessionData/{ProductType}/{DeviceId}/{FolderTimestamp}/...`
- Transaction data: `Transaction/{COPY|SCAN|PRINT|FAX}/{jobUuid}.transaction`
- Scan cache/data: `ScanTransferManager/cache.ndi` , `ScanMetadataTransferManager/cache.ndi` , `Scan/{ScanSessionId}/...`
- `{PersistenceRoot}/`
 - `HistoricalSessionData/`
 - `{ProductType}/`
 - `{DeviceId}/`
 - `{TimestampFolder}/` — session data files
 - `Transaction/`

- `COPY/{jobUuid}.transaction`
- `SCAN/{jobUuid}.transaction`
- `PRINT/{jobUuid}.transaction`
- `FAX/{jobUuid}.transaction`
- `LostTransaction/`
 - `{DeviceId}/COPY-{jobUuid}`
 - `{DeviceId}/SCAN-{jobUuid}`
 - `{DeviceId}/PRINT-{jobUuid}`
- `ScanTransferManager/cache.ndi`
- `ScanMetadataTransferManager/cache.ndi`
- `Scan/{scanSessionId}/` — temp scan image files

4) Session Data Cleanup (Time-Based)

- Scheduler: `HistoricalSessionManager` runs every 600 seconds.
- Expiration threshold: `now - 72h (calendar.add(Calendar.HOUR_OF_DAY, -24 * 3))`.
- Folder expires when `mostRecentTime < expirationDateTime`.
- This is not FIFO/LRU.

File lifecycle:

- Files created: `HistoricalSessionData/{ProductType}/{DeviceId}/{timestamp}/...` — written on `PersistHistoricalSessionDataRequest` (triggered on login / account / accounting form / workflow / escrow events)
- Files NOT deleted on logout: disk files are **not removed** on logout — only in-memory state is cleared (see logout flow below)
- Files deleted: `HistoricalSessionData/{ProductType}/{DeviceId}/{timestamp}/` — deleted by scheduled task every 600s if `mostRecentTime < now - 72h`
- Parent device root folder is also removed if it becomes empty after cleanup

Event chain P1 — Session persist (login → P1.5 write session data to disk)

P1.1 – Login / account / workflow / escrow event → post `SerializeTransactionSession`

`ISessionEvents.OnLoginEvent`

→ `HistoricalSessionManager.onEvent(OnLoginEvent)`

[Historic

```

→ processEvent(OnLoginEvent) [Histori
  → updateFakeJobForTransactionBase(...) [Histori
    → addIntoMostRecentSessionIds(session) [Histori
      [track sessionId in most-recent list (max 10)]
    → getFakeCopyJob() or getFakeScanJob() [create
    → post SerializeTransactionSessionBaseRequest [→ cont

```

IAccountEvents.LoginWithAccount (optional, fired when account is selected)

```

→ HistoricalSessionManager.onEvent(LoginWithAccount) [Historic
  → processEvent(LoginWithAccount) [Histori
    → historicalSessionManagerStateProvider.setDeviceAccount(account)
    → updateFakeJobForTransactionBase(...) [Histori
    → post SerializeTransactionSessionBaseRequest [→ cont

```

IAccountingCodeSessionEvents.LoginWithAccountingForm (optional, fired when account

```

→ HistoricalSessionManager.onEvent(LoginWithAccountingForm) [Historic
  → processEvent(LoginWithAccountingForm) [Historic
    → historicalSessionManagerStateProvider.setDeviceAccountingForm(accountin
    → updateFakeJobForTransactionBase(...) [Histori
    → post SerializeTransactionSessionBaseRequest [→ cont

```

IWorkflowEvents.BeginWorkflow / EndWorkflow (fired when a workflow starts or ends)

```

→ HistoricalSessionManager.onEvent(BeginWorkflow) [Historic
→ HistoricalSessionManager.onEvent(EndWorkflow) [Historic
  → processEvent(BeginWorkflow) [Histori
  → processEvent(EndWorkflow) [Histori
    → historicalSessionManagerStateProvider.setDeviceWorkflow(workflow) / rem
    → updateFakeJobForTransactionBase(...) [Histori
    → post SerializeTransactionSessionBaseRequest [→ cont

```

Copy.EscrowResponse (fired after copy escrow pre-authorization; carries escrowToke

```

→ HistoricalSessionManager.onEvent(EscrowResponse) [Historic
  → processEvent(EscrowResponse) [Histori
    → updateFakeJobForTransactionBase(..., referencedJobId=copyOemJobId, escr
    → post SerializeTransactionSessionBaseRequest [→ conti

```

P1.2 – SerializeTransactionSessionBaseRequest → posting provider serializes transac

SerializeTransactionSessionBaseRequest

```

→ EQ: TransactionJobPostingProvider.onEvent(SerializeTransactionSessionBaseReques
  → mapScanTransaction() or mapCopyTransaction() [EQ Tran
  → post SerializeTransactionSessionBaseResponseSuccess [→ conti
→ OM: OutputManagerTransactionJobPostingProvider.onEvent(SerializeTransactionSess
  → mapCopyTransaction() or mapScanTransaction() [OM Outp
  → [on success] post SerializeTransactionSessionBaseResponseSuccess [→ conti
  → [on failure] post SerializeTransactionSessionBaseResponseFailure

```

P1.3 – SerializeTransactionSessionBaseResponseSuccess → optional time sync

SerializeTransactionSessionBaseResponseSuccess

- HistoricalSessionManager.onEvent(SerializeTransactionSessionBaseResponseSuccess) [Historic
- processEvent(SerializeTransactionSessionBaseResponseSuccess) [Historic
- [if IDeviceTimeSyncController present]
- post DeviceConvertServerTimeRequest [convert
- [else]
- storeTransactionBase(event, null) [Historic

DeviceConvertServerTimeResponseSuccess (only when IDeviceTimeSyncController wired

- HistoricalSessionManager.onEvent(DeviceConvertServerTimeResponseSuccess) [Hist
- processEvent(DeviceConvertServerTimeResponseSuccess) [Historic
- storeTransactionBase(event, convertedTime) [Historic

P1.4 – storeTransactionBase() → add session data to in-memory map

- storeTransactionBase() [Historic
- new HistoricalSessionData(sessionId, transaction, referencedJobId, escrowToken)
- historicalSessionDataMap.put(finalDateTime, historicalSessionData) [add to
- historicalSessionManagerStateProvider.setHistoricalSessionData(...)
- persistHistoricalSessionData() [Historic
- post PersistHistoricalSessionDataRequest [→ conti

P1.5 – PersistHistoricalSessionDataRequest → write session data to disk

PersistHistoricalSessionDataRequest

- HistoricalSessionDataPersistenceHandler.onEvent(PersistHistoricalSessionDataReq
- onPersistRequestEvent() [Historic
- persistenceFactory.createFile(HistoricalSessionData/{ProductType}/{Devi
- [write session data to disk]
- post PersistHistoricalSessionDataResponse
- [no production subscriber – terminal event, used only in tests for ve

Event chain P2 — OnLogoutEvent → clear in-memory state (disk files NOT deleted)

P2.1 – OnLogoutEvent → clear in-memory state (disk files NOT deleted)

ISessionEvents.OnLogoutEvent

- HistoricalSessionManager.onEvent(OnLogoutEvent) [Historic
- processEvent(OnLogoutEvent) [Historic
- removeSessionEntries(deviceInstance) [Historic
- historicalSessionManagerStateProvider.removeDeviceAccount()

```

→ historicalSessionManagerStateProvider.removeDeviceWorkflow()
→ historicalSessionManagerStateProvider.removeDeviceAccountingForm()
→ historicalSessionManagerStateProvider.removeDeviceScanFormWithEvent
→ historicalSessionManagerStateProvider.removeDeviceNEUFScanSettingsF
→ removeFromMostRecentSessions()
    [if mostRecentSessionIds.size() > 10]
        → remove oldest sessionId from list
        → remove corresponding entries from in-memory historicalSessi
    [disk files are NOT deleted here – only in-memory state is cleared]

```

Event chain P3 — Session scheduled cleanup (time-based disk deletion every 600s)

P3.1 – Framework start → schedule cleanup task every 600s

```

Framework.onFrameworkStarted()
→ HistoricalSessionManager.onFrameworkStarted() [HistoricalSessionManager]
→ scheduledThreadPool.scheduleAtFixedRate(CleanUpTransactionBaseTask, 600s, 600s)

```

P3.2 – Every 600s: compute expiration time (now - 72h) → post remove request per device

```

[Every 600 seconds]
CleanUpTransactionBaseTask.run()
→ post OnClearHistoricalRecordsEventWrapper

OnClearHistoricalRecordsEventWrapper
→ HistoricalSessionManager.onEvent(OnClearHistoricalRecordsEventWrapper) [HistoricalSessionManager]
→ processEvent(OnClearHistoricalRecordsEventWrapper) [HistoricalSessionManager]
    → clearTransactionBaseRecords(deviceInstance) [HistoricalSessionManager]
        → calendar.add(HOUR_OF_DAY, -24 * 3) [timeOfExpiration]
        → for each device with session data:
            → currentDeviceInstancesForPersistentRemoval.add(deviceInstance)
            → postPersistenceRemoveRequest(timeOfExpiration) [HistoricalSessionManager]
                → currentDeviceInstancesForPersistentRemoval.remove(0) [processEvent]
                → post RemoveHistoricalSessionDataPersistenceRequest [→ continue]

```

P3.3 – RemoveHistoricalSessionDataPersistenceRequest → scan & delete expired folder

```

RemoveHistoricalSessionDataPersistenceRequest
→ HistoricalSessionDataPersistenceHandler.onEvent(RemoveHistoricalSessionDataPersistenceRequest) [HistoricalSessionDataPersistenceHandler]
→ onRemovePersistenceRequestEvent() [HistoricalSessionDataPersistenceHandler]
    → persistenceFactory.createFile(HistoricalSessionData/{ProductType}/{DeviceId})
    → for each folder in device persistence root:
        → persistenceFolderExpired(expirationDateTime, folderName) [HistoricalSessionDataPersistenceHandler]

```

```

        → read sessionInfo file from folder [format:
        → parse mostRecentTimeInMilliseconds
        [if mostRecentTime < expirationDateTime]
            → expired = true
    [if expired]
        → removePersistenceFolder() [delete
    [if any folder was deleted]
        → removePersistenceFolder(deviceRootPersistenceFolder) [remove e
    → post RemoveHistoricalSessionDataPersistenceResponse

```

5) Transaction Persistence (Event-Driven)

- Supported job types: Copy, Scan, Print, Fax.
- Active jobs stored in memory map (`mapRunTimeJobs`).
- Background task writes active jobs every 5 seconds.
- `RemovePersistenceRequest` is sent only after **transaction is successfully posted to business system**, not simply when the device job ends.
 - Triggered via `TransactionPersistenceController.deletePeristedJob()` .
 - For Scan: `ScanPersistenceController.onScanPostResponseSuccessEvent()` → `deletePeristedJob()` .
 - For Copy/Print: `TransactionJobPostingProvider.onEventAsync(*.PostRequest)` → post to server → on success → `deletePeristedJob()` .

Note: Fax job type is listed in storage paths but its event chain is not yet documented here.

Event chain per job type

Scan (P4 + P7):

Files created:

- `Transaction/SCAN/{jobUuid}.transaction` — transaction data, written every 5s, deleted on post success
- `ScanTransferManager/cache.ndi` — transfer recovery cache, deleted on `PushScanSuccess` / `PushScanCancel`
- `ScanMetadataTransferManager/cache.ndi` — metadata transfer cache, deleted on metadata transfer complete
- `Scan/{scanSessionId}/...` — scan image files, deleted on workflow end / `onFrameworkStopping()`

- `LostTransaction/{deviceId}/SCAN-{jobUuid}` — created only if posting fails after 3 retries

P4.1 – Scan.Begin → add transaction to in-memory map

`IDeviceJobEvents.Scan.Begin`

- `ScanPersistenceController.onScanBeginEvent()` [ScanPers
- `mapRunTimeJobs.put(jobUuid, transaction)` [add tran

P4.2 – Scan.Update → update transaction state in memory (repeats)

`IDeviceJobEvents.Scan.Update (repeat)`

- `ScanPersistenceController.onScanUpdateEvent()` [ScanPers
- `mapRunTimeJobs.replace(jobUuid, transaction)` [update t

P4.3 – Scan.SerializeEndSuccessRequest → write transaction to disk

`IDeviceJobEvents.Scan.SerializeEndSuccessRequest`

- `ScanPersistenceController.onScanSerializeEndSuccessResponseSuccessEvent()` [Sca
- `TransactionPersistenceController.persistJob()` [Transact
- `post ITransactionManager.Scan.PersistRequest`
- `TransactionPersistenceHandler.onEvent(Scan.PersistRequest)` [Transact
- `onPersistRequestEvent()` [Transac
- `mapRunTimeJobs.put(jobUuid, transaction)` [ensure j
- `persistenceFactory.createFile(Transaction/SCAN/{jobUuid}.transaction)`
- `write transaction to disk` [backgro
- `post ITransactionManager.Scan.PersistSuccess`
- [posting to backend → continue to P7.1]

P4.4 – [parallel] PushScanRequest → write cache.ndi + start file upload

[--- scan file transfer runs in parallel with P4.1–P4.3 above ---]

`IScanTransferManager.PushScanRequest`

- `ScanTransferManager.onEventAsync(PushScanRequest)` [ScanTran
- `validateRequest() + addManagerRequest()` [ScanTran
- `persistenceFactory.createFile(ScanTransferManager/cache.ndi)` [persistM
- [scan pages transfer in progress]

P4.5 – [parallel] Transfer result → delete cache files

[on transfer success]

- `IScanTransferManager.PushScanSuccess`
- `ScanTransferManager.onEventAsync(PushScanSuccess)` [ScanTran
- `checkManagerRequestState()`

```

        → delete ScanTransferManager/cache.ndi
        → delete ScanMetadataTransferManager/cache.ndi
[on transfer cancel]
→ IScanTransferManager.PushScanCancel
    → ScanTransferManager.onEventAsync(PushScanCancel) [ScanTran
        → delete cache files
[on framework stopping]
→ ScanManager.onFrameworkStopping()
    → scanStateController.cleanup()
        → delete all temp scan files under Scan/{scanSessionId}/
[--- end parallel scan file transfer ---]

```

P4.6 – Scan.PostRequest → post transaction to backend (→ then P7)

```

ITransactionJobPostingProvider.Scan.PostRequest
[EQ - Equitrac]
→ NEUF-Equitrac TransactionJobPostingProvider.onEventAsync(Scan.PostRequest) [EQ
    → WedpUtils.getProtocol()
    → container.getTransaction()
    → postWedpTransaction()
        → wedpProtocol.transactionPostDeviceJob(toPost) [EQ Trans
    → [on success] bus.post(Scan.PostResponseSuccess) [→ contin
    → [on failure: ServerOffline/Timeout/CommunicationLayerException]
        → bus.post(Scan.PostResponseFailure) [→ contin

```

Copy (P5 + P7):

Files created:

- `Transaction/COPY/{jobUuid}.transaction` — transaction data, written every 5s, deleted on post success
- `LostTransaction/{deviceId}/COPY-{jobUuid}` — created only if posting fails after 3 retries

P5.1 – Copy.EscrowRequest → serialize + write transaction to disk

```

IDeviceJoblogManager.OnJobLogEvent
→ [create IDeviceCopyJob]
IDeviceJobEvents.Copy.EscrowRequest
→ EQ: TransactionJobPostingProvider.onEventAsync(Copy.EscrowRequest) [EQ Trans
    → mapCopyTransaction() [EQ Tran
    → TransactionPersistenceController.persistJob() [Transact
    → post ITransactionManager.Copy.PersistRequest
    → TransactionPersistenceHandler.onEvent(Copy.PersistRequest) [Transact
        → onPersistRequestEvent() [Transac

```

```

        → mapRunTimeJobs.put(jobUuid, container) [Transact
        → [background task flushes every 5s] [write to
    → post ITransactionManager.Copy.PersistResponseSuccess
    → transactionPostEscrowDeviceJob()
→ IDeviceJobEvents.Copy.EscrowResponse
→ [Copy job completes on device]

```

P5.2 – Copy.PostRequest → post transaction to backend (→ then P7)

ITransactionJobPostingProvider.Copy.PostRequest

```

[EQ - Equitrac]
→ NEUF-Equitrac TransactionJobPostingProvider.onEventAsync(Copy.PostRequest) [EQ
    → WedpUtils.getProtocol()
    → container.getTransaction()
    → [if no escrow token] wedpProtocol.transactionPostDeviceJob(toPost)
    → [if escrow token] wedpProtocol.transactionPostReconcileDeviceJob(toPost,
    → [on success] bus.post(Copy.PostResponseSuccess) [→ contin
    → [on failure: ServerOffline/Timeout/CommunicationLayerException]
        → bus.post(Copy.PostResponseFailure) [→ contin

```

Print (P6 + P7):

Files created:

- `Transaction/PRINT/{jobUuid}.transaction` — transaction data, written every 5s, deleted on post success
- `LostTransaction/{deviceId}/PRINT-{jobUuid}` — created only if posting fails after 3 retries

P6.1 – Print.SerializeRequest → serialize + write transaction to disk

HTTP POST job log → PluginEventServlet.doPost()

```

    → IDeviceJoblogManager.OnJobLogEvent
    → [create IDevicePrintJob]

```

ITransactionJobPostingProvider.Print.SerializeRequest

```

→ EQ: TransactionJobPostingProvider.onEvent(Print.SerializeRequest) [EQ Trans
    → mapPrintTransaction() [EQ Tran
    → TransactionPersistenceController.persistJob() [Transact
    → post ITransactionManager.Print.PersistRequest
    → TransactionPersistenceHandler.onEvent(Print.PersistRequest) [Transact
        → onPersistRequestEvent() [Transac
            → mapRunTimeJobs.put(jobUuid, container) [Transact
            → [background task flushes every 5s] [write to
    → post ITransactionManager.Print.PersistResponseSuccess

```

P6.2 – Print.PostRequest → post transaction to backend (→ then P7)

ITransactionJobPostingProvider.Print.PostRequest

[EQ - Equitrac]

```
→ NEUF-Equitrac TransactionJobPostingProvider.onEventAsync(Print.PostRequest) [E
    → WedpUtils.getProtocol()
    → container.getTransaction()
    → [if overrideUuid]    wedpProtocol.transactionPostReconcileDeviceJob(toPost,
    → [if no overrideUuid] wedpProtocol.transactionPostDeviceJob(toPost) [EQ Tra
    → [on success] bus.post(Print.PostResponseSuccess) [→ contin
    → [on failure: ServerOffline/Timeout/CommunicationLayerException]
        → bus.post(Print.PostResponseFailure) [→ contin
```

Common shared flow: PostRequest flow P7 — PostRequest / retry / lost transaction (EQ / OM / Success / Failure)

Applies to **Scan** (from P4.6), **Copy** (from P5.2), **Print** (from P6.2).

P7.1 – {Scan/Copy/Print}.PostRequest → EQ/OM posts to backend server

ITransactionJobPostingProvider.{Scan/Copy/Print}.PostRequest

[OM - OutputManager]

```
→ NEUF-OutputManager OutputManagerTransactionJobPostingProvider.onEventAsync(Copy
→ NEUF-OutputManager OutputManagerTransactionJobPostingProvider.onEventAsync(Scan
→ NEUF-OutputManager OutputManagerTransactionJobPostingProvider.onEventAsync(Print
    → container.getTransaction()
    → converter.toXml(OutputDocumentStatuses.class, transaction)
    → outputManagerService.setJobStatusesWithXml(deviceInstance, xTransaction) [
    → [on success] bus.post({Scan/Copy/Print}.PostResponseSuccess) [→ contin
    → [on failure] bus.post({Scan/Copy/Print}.PostResponseFailure) [→ contin
```

P7.2 – PostResponseSuccess → delete persisted transaction file

[on {Scan/Copy/Print}.PostResponseSuccess]

```
→ ScanPersistenceController.onScanPostResponseSuccessEvent() [ScanPers
→ CopyPersistenceController.onCopyPostResponseSuccessEvent() [CopyPers
→ PrintPersistenceController.onPrintPostResponseSuccessEvent() [PrintPer
    → transactionRecords.getPostRequestById()
    → post ITransactionEvents.{Scan/Copy/Print}.PostSuccess
    → TransactionPersistenceController.deletePeristedJob() [Transact
    → post ITransactionManager.{Scan/Copy/Print}.RemovePersistenceRequest
    → TransactionPersistenceHandler.onEventAsync({Scan/Copy/Print}.RemovePersiste
        [Scan: TransactionPersistenceHandler.java:155]
        [Copy: TransactionPersistenceHandler.java:137]
```

```

[Print: TransactionPersistenceHandler.java:173]
→ onRemovePersistenceRequestEvent() [Transact
    → mapRunTimeJobs.remove(jobUuid) [Transact
    → removePersistedJob(jobUuid) [Transact
        → persistenceFactory.createFile(Transaction/SCAN|COPY|PRINT/{jobU
        → if (persistenceFile.exists()) { persistenceFile.delete() }
→ post ITransactionManager.{Scan/Copy/Print}.RemovePersistenceSuccess

```

P7.3 – PostResponseFailure (1st attempt) → hand off to LostTransactionsController f

```

[on {Scan/Copy/Print}.PostResponseFailure – first post, handled by {Scan/Copy/Pri
→ ScanPersistenceController (handled via LostTransactionsController)
→ CopyPersistenceController.onCopyPostResponseFailureEvent() [CopyPers
→ PrintPersistenceController.onPrintPostResponseFailureEvent() [PrintPer
    → transactionRecords.getPostRequestById()
    → post ITransactionManager.{Scan/Copy/Print}.RePostRequest [LostTran
    → post ITransactionEvents.{Scan/Copy/Print}.PostFailed
        [Copy – Android only] AndroidToastsManager.onEvent(ITransactionEvents.Cop
            → postPlatformBannerMessage(...)
        [Scan, Print] no subscriber in core
    → transactionRecords.remove(postRequestId, jobUuid)
→ LostTransactionsController.onEvent({Scan/Copy/Print}.RePostRequest)
    [Scan: LostTransactionsController.java:337]
    [Copy: LostTransactionsController.java:318]
    [Print: LostTransactionsController.java:356]
    → onRepostRequest() [LostTra
        → create new ITransactionJobPostingProvider.{Scan/Copy/Print}.PostRequest
        → transactionRecords.setRequestIdRetryTimes(postRequestId, 0)
        → scheduleNextPosting(TransactionRepostingTask) [LostTra

```

P7.4 – Retry PostResponseFailure (retryTimes < 3) → schedule next retry

```

[on retry {Scan/Copy/Print}.PostResponseFailure – retries handled by LostTransact
→ LostTransactionsController.onEvent({Scan/Copy/Print}.PostResponseFailure)
    [Scan: LostTransactionsController.java:350]
    [Copy: LostTransactionsController.java:331]
    [Print: LostTransactionsController.java:369]
    → onPostResponseFailureEvent() [LostTra
        → retryTimes = currentRetry + 1
        → [if bOffline] keep retryTimes unchanged
        → [if retryTimes < 3] scheduleNextPosting(TransactionRepostingTask) [Los

```

P7.5 – retryTimes >= 3 → rename transaction to LostTransaction file

```

→ [if retryTimes >= 3]
    → persistLostJob() [LostTra

```

```

→ post ITransactionManager.{Scan/Copy/Print}.PersistLostTransactionRe
→ TransactionPersistenceHandler.onEventAsync({Scan/Copy/Print}.Persis
  [Scan: TransactionPersistenceHandler.java:161]
  [Copy: TransactionPersistenceHandler.java:143]
  [Print: TransactionPersistenceHandler.java:179]
→ onPersistLostTransactionRequestEvent() [Transac
  → writeLostJob() [Transac
    → rename Transaction/SCAN|COPY|PRINT/{jobUuid}.transactio
      → LostTransaction/{deviceId}/SCAN|COPY|PRINT-{jobUu
    → mapRunTimeJobs.remove(jobUuid) or mapUnsentTransactions.rem
      [Transac
    → post ITransactionManager.{Scan/Copy/Print}.PersistLostTrans
→ post ITransactionEvents.{Scan/Copy/Print}.PostFailed
  [Copy – Android only] AndroidToastsManager.onEvent(ITransactionEvents
    → postPlatformBannerMessage(...)
  [Scan, Print] no subscriber in core

```

6) Lexmark Integration

- `NEUF-LexmarkUC-Platform/.../PersistenceFactory.java` is a wrapper implementation of `IPersistenceFactory`.
 - It handles file creation and recursive delete utility.
 - Cleanup policy remains from DWS core handlers (session/transaction/scan).
 - `Lexmark PersistenceFactory` (*createFile / streams / deleteFile*)
 - implements `IPersistenceFactory` contract
 - used by **DWS Core Handlers**:
 - `HistoricalSessionManager` + `HistoricalSessionDataPersistenceHandler`
 - 3-day time-based cleanup
 - `TransactionPersistenceHandler` → event-based cleanup
 - `ScanTransferManager` → explicit scan cleanup
-